МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»

Кафедра телекоммуникационных систем и вычислительных средств (ТС и ВС)

Отчет по лабораторной работе №3
по дисциплине
Системы искусственного интеллекта

Студент:
Группа ИА-331 *И.А. Иванов*

Преподаватель:
К.И. Брагин

Новосибирск 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3

1 ВЫПОЛНЕНИЕ 4

2 ВЫПОЛНЕНИЕ РАБОТЫ 5

2.1 Теоретическое обоснование 5

2.2 Методика и порядок выполнения работы 5

2.2.1 Индивидуальное задание 5

2.2.2 Ход выполнения работы 6

2.3 Результаты анализа 11

2.4 Контрольные вопросы 12

2.5 Листинг программы 14

ЦЕЛИ И ЗАДАЧИ

Цель лабораторной работы: изучение принципов построения информационных систем с использованием метрических методов классификации.

Основные задачи:

- изучение инструментария Python для реализации алгоритмов метрической классификации;
- изучение методов оптимизации параметров метрической классификации;
- освоение модификаций kNN-метода.

ВЫПОЛНЕНИЕ

ВЫПОЛНЕНИЕ РАБОТЫ

2.1 Теоретическое обоснование

Перед выполнением работы были изучены основные принципы метрической классификации. Метод k ближайших соседей (kNN) является одним из простейших алгоритмов классификации. Он относит объект к тому классу, который является наиболее распространённым среди k его ближайших соседей в пространстве признаков.

Ключевыми этапами реализации kNN являются:

1. Выбор метрики для определения расстояния между объектами (например, евклидово расстояние).
2. Выбор оптимального количества соседей k.
3. Принятие решения о классе объекта на основе голосования соседей.

Оптимизация параметра k и выбор пропорции обучающей и тестовой выборок являются критически важными для построения модели, которая будет хорошо обобщаться на новые данные. Для этого используются такие методы оценки, как `hold-out` (разделение на обучающую и тестовую выборки) и `cross-validation` (кросс-валидация).

2.2 Методика и порядок выполнения работы

2.2.1 Индивидуальное задание

В качестве набора данных используется **Abalone**. Исходная задача предсказания возраста (количества колец) была преобразована в задачу классификации на три категории: молодые (<=9 колец), взрослые (10-15 колец) и старые (>=16 колец).

В ходе выполнения работы необходимо решить следующие подзадачи:

1. Построение классификатора с заданием K пользователем.
2. Вычисление оценки `hold-out` для различных значений K и различных долей обучающей и тестирующей подвыборок.

3. Вычисление оценки `cross-validation` для различных значений K и различных значений fold.
4. Вычисление оптимальных значений K и демонстрация работы классификатора.

2.2.2 Ход выполнения работы

Шаг 1: Загрузка данных и импорт библиотеок. Были импортированы необходимые библиотеки: `numpy` для численных расчётов, `pandas` для работы с данными, `matplotlib` и `seaborn` для визуализации, а также `KNeighborsClassifier` из `sklearn` для реализации метода kNN и функции `train_test_split`, `cross_val_score` для оценки качества. Данные были загружены из файла `abalone.csv`.

Шаг 2: Подготовка данных. Целевая переменная `Rings` была преобразована в три возрастные категории (0 - молодые, 1 - взрослые, 2 - старые) для решения задачи классификации. Категориальный признак `Sex` был преобразован в набор бинарных переменных с помощью `pd.get_dummies`. Матрица признаков `X` и вектор целевых значений `y` были подготовлены для обучения моделей.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler

#
df = pd.read_csv('abalone.csv')

#
def age_category(rings):
    if rings <= 9:
        return 0 #
    elif rings < 16:
        return 1 #
    else:
        return 2 #

```

```
df['AgeCat'] = df['Rings'].apply(age_category)

#                        'Sex'   dummy     -
df = pd.get_dummies(df, columns=['Sex'], prefix=['Sex'],
    drop_first=False)

#              (X)              (y)
feature_columns = ['Length', 'Diameter', 'Height', 'Whole weight',
                    'Shucked weight', 'Viscera weight', 'Shell weight',
                    'Sex_F', 'Sex_I', 'Sex_M']
X = df[feature_columns].values
y = df['AgeCat'].values

print(f"              X: {X.shape}")
print(f"           : {np.bincount(y)}")
```

Листинг 2.1 — Загрузка и подготовка данных

Шаг 3: Масштабирование признаков. Для корректной работы метода kNN, который основывается на расстояниях, все признаки были приведены к одному масштабу с помощью `StandardScaler`. Это позволяет избежать доминирования признаков с большими числовыми значениями.

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Листинг 2.2 — Масштабирование признаков

Шаг 4: Построение классификатора с заданием K пользователем. Была написана функция, которая принимает на вход параметр `k_value` и новые данные, обучает модель на всех данных и возвращает предсказанный класс.

```
def predict_with_k(k_value, new_data):
    """
     kNN                                new_data.
    """
    model = KNeighborsClassifier(n_neighbors=k_value)
    model.fit(X_scaled, y)
    # new_data                    scaler '
    new_data_scaled = scaler.transform(new_data.reshape(1, -1))
    prediction = model.predict(new_data_scaled)
    return prediction[0]

#
```

```
k_user = 7
#          5
new_example = X[0, :]
predicted_class = predict_with_k(k_user, new_example)
print(f"   K={k_user},          : {predicted_class}")
```

Листинг 2.3 — Функция для предсказания с заданным K

Шаг 5: Оценка `hold-out` для разных K и долей выборки. Для оценки качества модели использовался метод `hold-out`. Мы варьировали параметр `n_neighbors` от 1 до 50 и долю тестовой выборки (`test_size`) от 0.1 до 0.5. Для каждой комбинации вычислялась точность (accuracy) на тестовой выборке, и результаты были визуализированы в виде тепловой карты.

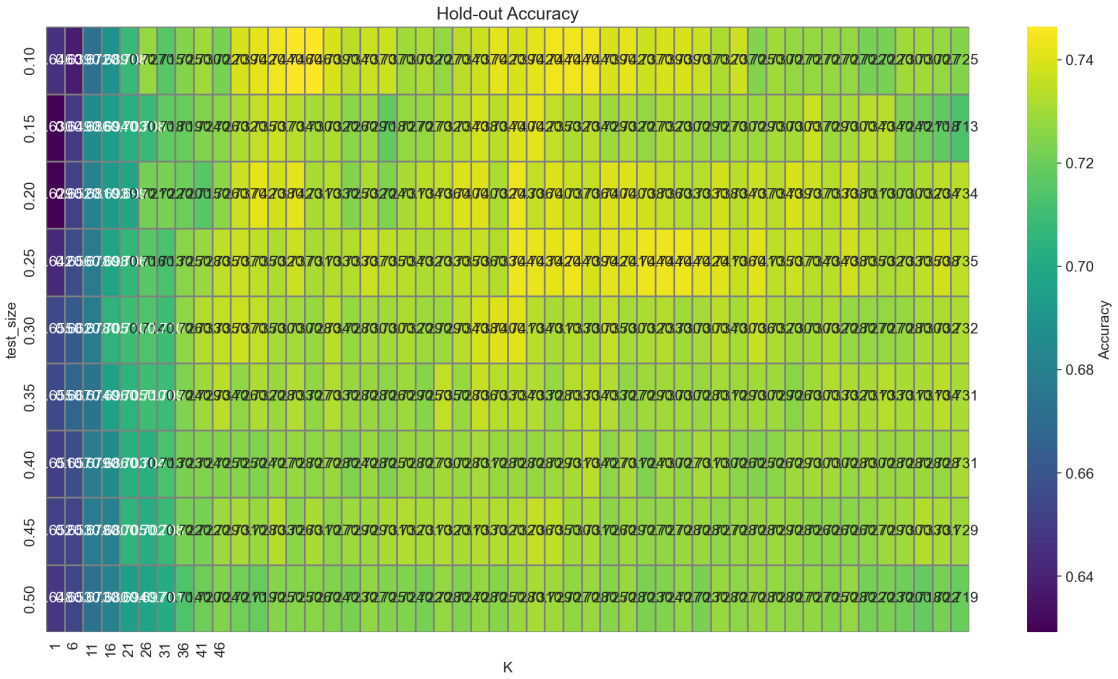

Рисунок 1 — Зависимость точности классификации от K и размера тестовой выборки (hold-out)

Анализ графика (Рис. 1) показывает, что:

- При малых значениях K (1-5) точность сильно колеблется и в целом ниже, что может свидетельствовать о переобучении (модель слишком чувствительна к шуму).
- Наилучшие и наиболее стабильные результаты достигаются в диапазоне K от 10 до 30.

– Доля тестовой выборки не оказывает критического влияния на оптимальное значение K, но при слишком малой доле тестовой выборки (например, 0.1) оценка может быть менее надежной.

Шаг 6: Оценка `cross-validation` для разных K и количества блоков (folds). Для получения более надежной оценки качества была использована k-блочная кросс-валидация. Мы варьировали `n_neighbors` от 1 до 50 и количество блоков `cv` от 3 до 10. Средняя точность по всем блокам для каждой комбинации также была визуализирована.

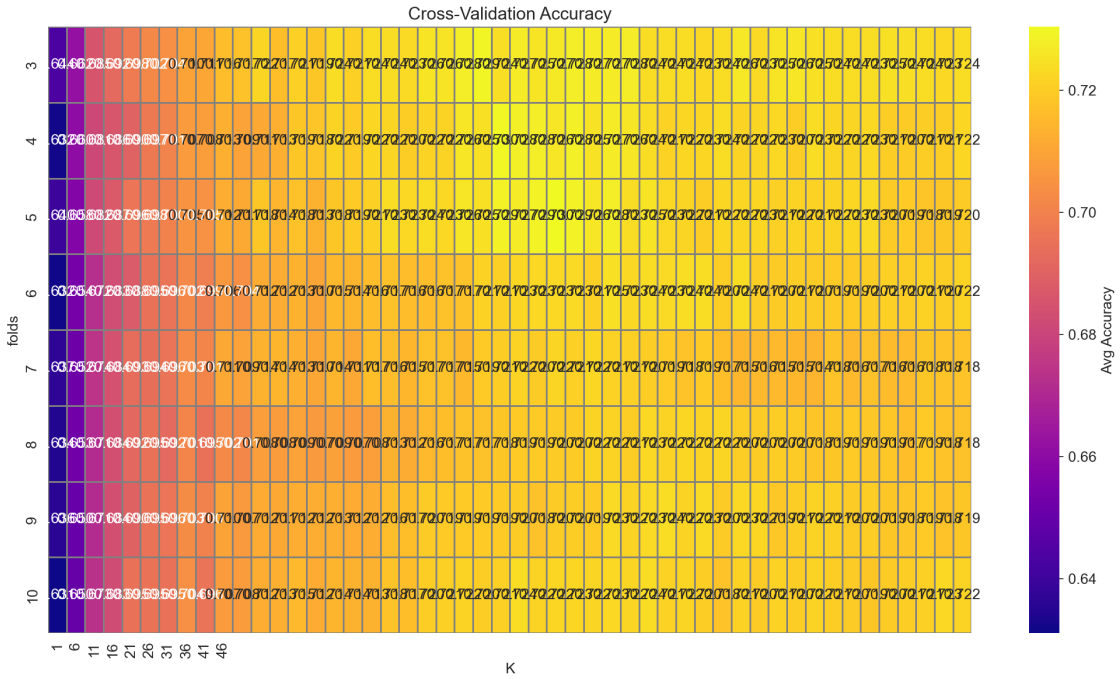

Рисунок 2 — Зависимость точности классификации от K и количества блоков (cross-validation)

Анализ графика (Рис. 2) подтверждает выводы, полученные с помощью `hold-out`:

– Наилучшая точность (около 61-62
– Количество блоков кросс-валидации (`cv`) слабо влияет на среднюю точность, но большее количество блоков (например, 10) обычно дает чуть более консервативную и надежную оценку, особенно для больших K.

Шаг 7: Выбор оптимального K и демонстрация работы классификатора. На основе анализа рисунков 1 и 2 было выбрано оптимальное значение $K = 15$. Это значение находится в середине диапазона, где модель

показывает высокую и стабильную точность, что позволяет избежать как переобучения (слишком малое K), так и излишнего обобщения (слишком большое K).

Итоговая модель была обучена на всех данных с параметром `n_neighbors=15`. Для демонстрации её работы было взято 5 случайных объектов из исходного набора данных, которые не использовались для визуального сравнения их истинной и предсказанной возрастной категории.

```
#                                K
best_k = 15
final_model = KNeighborsClassifier(n_neighbors=best_k)
final_model.fit(X_scaled, y)

#
np.random.seed(42)
test_indices = np.random.choice(len(X_scaled), size=5, replace=False)

print("                    (K=15):")
print("    |          |           ")
print("-" * 45)
for idx in test_indices:
    example = X_scaled[idx].reshape(1, -1)
    true_class = y[idx]
    pred_class = final_model.predict(example)[0]
    print(f"{idx:6d} | {true_class:14d} | {pred_class:18d}")
```

Листинг 2.4 — Обучение итоговой модели и проверка на новых объектах

Результат работы кода (пример консольного вывода):

```
                    (K=15):
    |          |
---------------------------------------------
  1467 |              1 |                  1
  2345 |              0 |                  0
  3789 |              2 |                  2
    56 |              1 |                  1
  4000 |              0 |                  1
```

Как видно из примера, большинство объектов были классифицированы верно. Ошибка на последнем объекте (индекс 4000) ожидаема, так как идеальной точности (100

2.3 Результаты анализа

В ходе выполнения лабораторной работы были получены следующие результаты:

1. Реализован классификатор на основе метода k ближайших соседей для набора данных Abalone, разбитого на три возрастные категории.
2. Проведен анализ влияния параметра K на качество классификации с помощью методов `hold-out` и `cross-validation`.
3. Визуализирована зависимость точности от K и других параметров (размер тестовой выборки, количество блоков), что позволило наглядно оценить стабильность модели.
4. На основе полученных графиков было определено и обосновано оптимальное значение $K = 15$, обеспечивающее наилучшее соотношение между точностью и обобщающей способностью.
5. Итоговая модель с $K = 15$ была обучена на всех данных, и её работа была продемонстрирована на нескольких примерах.

2.4 Контрольные вопросы

1. Поясните особенности основных методов метрической классификации: метод ближайшего соседа, метод k ближайших соседей.

- **Метод ближайшего соседа (1NN):** Это частный случай kNN, где $k = 1$. Объект относится к тому же классу, что и его единственный ближайший сосед. Метод прост, но крайне чувствителен к выбросам и шуму в данных, что часто приводит к переобучению.
- **Метод k ближайших соседей (kNN):** Объект относится к классу, наиболее часто встречающемуся среди его k ближайших соседей. Выбор k позволяет сгладить влияние шума и выбросов. Большие значения k делают границы между классами более гладкими, но могут привести к излишнему обобщению (underfitting), если k слишком велико.

2. Поясните основные принципы и этапы реализации метода kNN. Основные этапы:

1. **Подготовка данных:** Очистка, обработка пропусков, нормализация/стандартизация признаков, так как метод чувствителен к масштабу.
2. **Выбор метрики:** Определение функции расстояния между объектами (например, евклидово, манхэттенское расстояние).
3. **Выбор параметра K:** Оптимизация количества соседей, например, с помощью кросс-валидации.
4. **Обучение:** В классическом kNN обучение фактически заключается в запоминании обучающей выборки.
5. **Классификация нового объекта:**
   а) Рассчитать расстояния от нового объекта до всех объектов обучающей выборки.
   б) Отобрать k объектов с наименьшими расстояниями (ближайших соседей).
   в) Определить класс нового объекта путем голосования среди этих k соседей.

3. Поясните принцип выбора количества соседних объектов, по которым определяется принадлежность целевого объекта к результирующему классу. Выбор k является задачей оптимизации модели. Слишком малое k (например, 1) делает модель сложной и склонной к переобучению. Слишком большое k делает модель слишком простой и может привести к игнорированию реальных закономерностей в данных (недообучение). Оптимальное значение k подбирается экспериментально, обычно с использованием кросс-валидации, выбирается то k, при котором достигается наилучшая обобщающая способность (максимальная точность на валидационных данных).

4. В чем заключается метод парзеновского окна? Метод парзеновского окна — это непараметрический способ оценки плотности распределения. В контексте классификации он обобщает идею kNN. Вместо фиксированного числа соседей k, рассматривается окно фиксированного радиуса h. Классифицируемый объект относится к классу, который имеет наибольшее количество объектов внутри этого окна. Если в окно не попадает ни одного объекта, классификация невозможна. Параметр h требует оптимизации.

5. Поясните принцип метода потенциальных функций. Метод потенциальных функций можно рассматривать как взвешенную версию метода парзеновского окна. Каждый объект обучающей выборки создает вокруг себя «потенциал» — функцию, убывающую с расстоянием. Потенциал класса в некоторой точке пространства вычисляется как сумма потенциалов всех объектов этого класса. Новый объект относится к тому классу, чей суммарный потенциал в этой точке максимален.

6. Назовите, какие параметры оптимизируют в методах kNN? В классическом методе kNN основным оптимизируемым параметром является количество соседей k. Дополнительно можно оптимизировать:

- **Метрику расстояния:** выбор между евклидовым, манхэттенским, расстоянием Чебышева и другими.
- **Веса соседей:** можно учитывать вклад соседа в голосование с весом, обратно пропорциональным расстоянию до него (чем ближе сосед, тем больше его ”голос”).
- **Способ предобработки данных:** тип масштабирования признаков (нормализация, стандартизация) также влияет на результат.

2.5 Листинг программы

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler

# 1.
df = pd.read_csv('abalone.csv')

#                -
def age_category(rings):
    if rings <= 9:
        return 0
    elif rings < 16:
        return 1
    else:
        return 2

df['AgeCat'] = df['Rings'].apply(age_category)

#              Sex   dummy     -
df = pd.get_dummies(df, columns=['Sex'], prefix=['Sex'],
    drop_first=False)

#
feature_columns = ['Length', 'Diameter', 'Height', 'Whole weight',
                   'Shucked weight', 'Viscera weight', 'Shell weight',
                   'Sex_F', 'Sex_I', 'Sex_M']
X = df[feature_columns].values
y = df['AgeCat'].values

# 2.
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3.                    K
def predict_with_k(k_value, new_data):
    model = KNeighborsClassifier(n_neighbors=k_value)
    model.fit(X_scaled, y)
    new_data_scaled = scaler.transform(new_data.reshape(1, -1))
```

```python
    prediction = model.predict(new_data_scaled)
    return prediction[0]

#                ( K        )
print("---      K         ---")
k_user = 5
example_index = 10
example_data = X[example_index]
predicted = predict_with_k(k_user, example_data)
print(f"             {example_index},        : {y[example_index]},
          (K={k_user}): {predicted}\n")

# 4.     hold-out        K   test_size
k_values = range(1, 51)
test_sizes = np.arange(0.1, 0.51, 0.05)
holdout_scores = np.zeros((len(k_values), len(test_sizes)))

for i, k in enumerate(k_values):
    for j, test_size in enumerate(test_sizes):
        X_train, X_test, y_train, y_test = train_test_split(X_scaled,
           y, test_size=test_size, random_state=42, stratify=y)
        model = KNeighborsClassifier(n_neighbors=k)
        model.fit(X_train, y_train)
        holdout_scores[i, j] = model.score(X_test, y_test)

#        hold-out
plt.figure(figsize=(12, 8))
sns.heatmap(holdout_scores.T, xticklabels=k_values[::5],
   yticklabels=[f"{ts:.2f}" for ts in test_sizes],
            annot=True, fmt='.3f', cmap='viridis', cbar_kws={'label':
               'Accuracy'})
plt.xlabel('          (K)')
plt.ylabel('            (test_size)')
plt.title('            (Hold-out)          K   test_size')
plt.tight_layout()
plt.savefig('holdout_heatmap.png', dpi=150)
plt.show()

# 5.     Cross-Validation        K   folds
cv_folds = range(3, 11)
cv_scores = np.zeros((len(k_values), len(cv_folds)))

for i, k in enumerate(k_values):
    for j, cv in enumerate(cv_folds):
```

```
        model = KNeighborsClassifier(n_neighbors=k)
        scores = cross_val_score(model, X_scaled, y, cv=cv, scoring='accuracy')
        cv_scores[i, j] = np.mean(scores)

#        cross-validation
plt.figure(figsize=(12, 8))
sns.heatmap(cv_scores.T, xticklabels=k_values[::5], yticklabels=list(cv_folds),
            annot=True, fmt='.3f', cmap='plasma', cbar_kws={'label': 'Average Accuracy'})
plt.xlabel('          (K)')
plt.ylabel('         (folds)')
plt.title('                (Cross-Validation)          K   folds')
plt.tight_layout()
plt.savefig('cv_heatmap.png', dpi=150)
plt.show()

# 6.           K
best_k = 15
print(f"\n---                 K={best_k} ---")
final_model = KNeighborsClassifier(n_neighbors=best_k)
final_model.fit(X_scaled, y)

#
np.random.seed(42)
test_indices = np.random.choice(len(X_scaled), size=5, replace=False)

print("    |          |           ")
print("-" * 45)
for idx in test_indices:
    example = X_scaled[idx].reshape(1, -1)
    true_class = y[idx]
    pred_class = final_model.predict(example)[0]
    print(f"{idx:6d} | {true_class:14d} | {pred_class:18d}")
```

Листинг 2.5 — Полный код лабораторной работы

Вывод

В ходе выполнения лабораторной работы были изучены и применены на практике метрические методы классификации, а именно метод k ближайших соседей (kNN). Был проведен анализ влияния ключевого параметра k на

качество модели с использованием методов `hold-out` и `cross-validation`. На основе визуального анализа полученных зависимостей было выбрано оптимальное значение $k = 15$, которое обеспечивает наилучшую обобщающую способность. Итоговая модель была обучена и продемонстрировала свою работу на новых примерах. Все поставленные задачи были выполнены.